SDU University
School of Information Technology and Applied Mathematics

DIPLOMA PROJECT

ARIA. The personal AI assistant for home monitoring, device control, and information access.

Kenesbayev Arsen, Orynbek Aidos, Sagibek Adil

6B06102 - "Computer Science"

Kaskelen, 2026

SDU University

School of Information Technology and Applied Mathematics

Dean of Faculty
Assistant Professor,
Ph.D. Ramis Akhmedov

_____

« » 2026y.

DIPLOMA PROJECT

ARIA. The personal AI assistant for home monitoring, device control, and information access.

6B06102 - "Computer Science"

Supervisor: Binara Imankulova

Students: Kenesbayev Arsen, Orynbek Aidos, Sagibek Adil

Kaskelen, 2026

Abstract

This thesis project describes the development of an intelligent personal assistant that is designed for home management and information access. While modern smart home technologies offer significant convenience, consolidating these features into one system with many diverse functionalities, such as remote control, everyday applications(weather, music, email), and low-latency STT/TTS techniques for multiple languages (especially Kazakh) is still a challenge. To address this issue, this thesis discusses present-day techniques used in AI and robotics and implements them as a 3D-printed unit that is controlled by ESP32, with a Flask server functioning as the central processing hub.

Аңдатпа

Бұл дипломдық жобада үйді басқаруға және ақпаратқа қол жеткізуге арналған зияткерлік жеке көмекшіні әзірлеу сипатталған. Заманауи «ақылды үй» технологиялары айтарлықтай ыңғайлылықты ұсынғанымен, бұл мүмкіндіктерді қашықтан басқару, күнделікті қолданбалар (ауа райы, музыка, электрондық пошта) және бірнеше тілге (әсіресе қазақ тіліне) арналған төмен кідірісті STT/TTS (сөйлеуді мәтінге және мәтінді сөйлеуге айналдыру) технологиялары сияқты көптеген әртүрлі функцияларды бір жүйеге біріктіру әлі де күрделі мәселе болып қалуда. Осы мәселені шешу үшін бұл жұмыста жасанды интеллект пен робототехникада қолданылатын қазіргі заманауи әдістер қарастырылады және олар Flask сервері орталық өңдеу торабы ретінде қызмет ететін, ESP32 арқылы басқарылатын 3D-принтерде басып шығарылған құрылғы түрінде жүзеге асырылады.

Аннотация

В данном дипломном проекте описывается разработка интеллектуального персонального помощника, предназначенного для управления домом и доступа к информации. Хотя современные технологии «умного дома» обеспечивают значительное удобство, объединение этих возможностей в единую систему с множеством разнообразных функций - таких как дистанционное управление, повседневные приложения (погода, музыка, электронная почта) и технологии STT/TTS с низкой задержкой для нескольких языков (в особенности казахского) - по-прежнему остается сложной задачей. Для решения этой проблемы в данной работе рассматриваются современные методы, применяемые в области искусственного интеллекта и робототехники, и реализуются в виде напечатанного на 3D-принтере устройства под управлением микроконтроллера ESP32, где сервер Flask функционирует в качестве центрального узла обработки данных.

Contents

Chapter 1

Introduction

1.1 Problem Background

The popularity of home automation technologies has grown significantly over the past decade. These technologies are used to improve comfort and entertainment. Here's a simple example: In many modern residential complexes today, developers are installing access systems and other automated solutions. Devices, such as lighting systems, voice assistants and monitoring systems, can work together thanks to these technologies. All equipment can be controlled via a smartphone provided it is connected to the network [1].

Despite the widespread adoption of home automation systems, users may face challenges. Since many home automation platforms are based on closed systems, devices from different manufacturers can be incompatible. Installation can also be challenging; users should consider whether they plan to add automation devices to their home in advance, as some complex devices require structural modifications.

It is important to remember that all devices must be integrated for proper and seamless operation. Integrating streaming video, voice functions, device control and web services requires extensive networking and programming knowledge. Of course, there are off-the-shelf products that are ready to use right away, but these limit users' ability to customise them. In regions where multilingual communication is commonplace, the possibility of supporting multiple languages should be considered.

To overcome these obstacles, this project focuses on developing a local 'smart home' system called ARIA. This system connects the web application to the hardware components deployed on the local network, enabling users to control their "smart" devices via a browser window or voice commands. Leveraging modern web technologies and APIs has enabled the creation of a unified system capable of controlling lighting fixtures, cameras, weather information and email [2, 3, 4, 5].

This solution has been implemented as a functional prototype on a local network. Main communication is demonstrated between a Flask web server [2] installed on a personal computer and embedded systems based on ESP32 microcontroller modules [6]. Additionally, a physical robot

equipped with a camera and status indicator lights serves as an example of the system's hardware interface.

1.2 Aim and Objectives

Aim. The project's goal is to create a system that is called ARIA, a smart home assistant that allows users to control devices and access various services via a web interface with voice interaction capabilities. ARIA bridges the gap between software and hardware.

Objectives. The objectives of this project:

- Developing a system control panel that provides functions related to device control, system monitoring and camera viewing via web development frameworks.
- Implementing a smart lighting controller that supports power management, brightness adjustment and colour customisation.
- Enabling video streaming from the system's camera module using embedded device programming methods.
- Providing voice control capabilities through speech recognition and text-to-speech conversion utilising machine learning approaches.
- Integrating external information services, such as weather, email and media services using web APIs and authentication procedures.

1.3 Thesis Outline

The first chapter of the document is an Introduction in which we described the aim and objectives of our project.

The second chapter provides an overview of the relevant technologies and existing approaches to smart home systems, voice assistants, and Internet of Things (IoT) devices. The software and hardware resources used in the project development are presented. Specifically, this includes web frameworks, communication interfaces, and embedded platforms.

The third chapter provides information about the design and architecture of the developed system. It gives the description of the web application structure, data exchange among the system modules, hardware configuration, and user interface design.

The fourth chapter describes the development and evaluation of the ARIA system. It details the process of integration of hardware and software modules, the system configuration, and testing.

Finally, a conclusion presents the outcomes of the project, potential limitations, and further improvement directions.

Chapter 2

Literature Review

Artificial intelligence has advanced far faster than anticipated five years ago, enabling applications from medical diagnostics to voice-controlled lighting. The challenge is no longer whether AI can be useful, but how to combine the right tools into a system that works reliably outside a demo. ARIA addresses exactly this: it integrates an LLM, speech recognition, speech synthesis, hardware, and web services into an open-source smart-home assistant. This chapter reviews the key technologies ARIA relies on and closes with a comparative analysis against existing products.

2.1 Large Language Models and Speech Technologies

Our conception of conversational assistants has been greatly influenced by the development of large language models (LLMs). First-generation systems like Alexa, Google Assistant, and Siri relied on an *intent-and-slot* pipeline where utterances were matched against a predefined set of intents and named entities were extracted. It worked great for a pre-determined list of commands but broke down under shifting phraseology and expectations for more reasoned responses.

LLMs make this approach obsolete. Google's Gemini [14], OpenAI's GPT-4 [15], and Meta's LLaMA [16] are trained on hundreds of billions of tokens and provide flexible, free-form responses. When choosing an LLM for ARIA, we focused on three metrics: hosting availability, latency and cost, and multilingual support. On the topic of hosting, Gemini and GPT-4 are available through cloud APIs while LLaMA is open-weight but requires 16-24 GB of VRAM on the host device, thereby making most consumer laptops ineligible. Regarding latency and cost, Gemini Flash supports sub-second first-token latency and even a free tier, GPT-4 is slower and metered, and LLaMA incurs no cost but offloads it to hardware. With regard to multilingual support, both Gemini and GPT-4 work well for English, Russian, and Kazakh, while the open LLaMA checkpoints give noticeably worse results in Kazakh. Based on this, Gemini was selected as the main reasoning model.

Every LLM has a certain cutoff for its knowledge base, and to address that, RAG was introduced by Lewis et al. [17]: relevant documents are retrieved from an external source and included into the prompt prior to generation. Our approach is based on an abbreviated version of this idea where, whenever a request concerns the user's mailbox, cached Gmail content and a short

context-memory block are added to the prompt.

In evaluating speech recognition algorithms, three solutions were considered. Firstly, cloud APIs (provided by Google, Azure, and Amazon Transcribe) yield good results but are metered and need Internet access. Secondly, Whisper [10], trained on 680k hours of data, is robust to accent and noise but delivers real-time performance only on a GPU. Thirdly, the Faster-Whisper [11] reimplementation of Whisper with CTranslate2 and INT8 quantization achieves up to 4× speed-ups and real-time performance on CPU alone, even enabling streaming. As Whisper struggles with Kazakh, we utilize *Mangisoz STT* as the main Kazakh engine, Faster-Whisper for English and Russian, and Gemini-powered recognition for fallback purposes.

As for speech synthesis, we prefer the Microsoft Edge TTS [12] service over the Google Cloud TTS and Amazon Polly services (both billed), as well as ElevenLabs (with limited free tiers) due to Edge TTS being free and keyless, supporting neural voices for all our target languages. The audio format conversion between the ESP32 audio node (16-bit / 16 kHz PCM) and browser/server-side formats is handled by pydub [18].

2.2 Web Framework, Data Layer, and API Integration

Three commonly used Python web frameworks are Django [19], FastAPI [20], and Flask [2]. Django is a comprehensive framework with its built-in ORM and admin site, but the heavy monolithic design is too much for a single-household setup. FastAPI is an excellent ASGI solution that fits well into RESTful microservices. Flask is a small micro-framework with extendability options that allow precise customization for a particular case. However, the critical factor when choosing Flask was the need for a full-duplex communication channel: regular HTTP is too slow for streaming audio in real time, while Flask-SocketIO [7] - the Flask package wrapping the Socket.IO [8] protocol - provides full-duplex WebSocket capability with long-polling fallback that was absent in both Django and FastAPI. Therefore, Flask was chosen.

Regarding persistence, SQLite [21] was chosen over PostgreSQL since ARIA works on a small single-household scale: SQLite stores its database in a single file, runs entirely in-process, and, unlike PostgreSQL, has no problems with concurrency at this scale. Database interaction is carried out via SQLAlchemy [22], wrapped by the Flask-SQLAlchemy [23] package. The Flask-Limiter [24] package limits requests per IP per minute to protect the app from exhausting Gemini API request quotas.

The most challenging integration comes from external services. The weather data are pulled from the OpenWeatherMap API [3], and searching functionality is implemented with REST calls to the YouTube Data API [5]. To authenticate to Gmail [4] via OAuth 2.0 [25], the whole authorization-code-flow implementation with server-side token storage and auto-refresh [13] is required to make a user sign in only once. For all outbound HTTP requests the Requests [26]

package is used, with credentials stored using python-dotenv [27].

2.3 Embedded Systems and IoT Control

The most prominent contenders for the voice-controlled Wi-Fi robot microcontroller include the Raspberry Pi series, Arduino, and the ESP32 by Espressif. Raspberry Pi runs the Linux OS and supports machine learning locally, but its cost, power draw, and boot time are too high for a peripheral node. Traditional Arduinos have neither built-in Wi-Fi nor DMA-capable I2S needed to stream sound at 16 kHz. ESP32 sports an Xtensa dual-core processor, Wi-Fi and Bluetooth, multiple DMA-capable I2S busses, a well-established software package (ESP-IDF [6]), a very cheap price tag and instant boot-up, thus qualifying as an appropriate choice of microcontroller for ARIA hardware nodes.

In ARIA, the ESP32-WROOM serves as an audio full-duplex bridge, sending packets containing PCM audio from the INMP441 microphone using UDP to the server while playing audio coming in on a different port through a MAX98357 Class-D amplifier on the I2S TX interface. Using a completely digital audio chain allows avoiding the background noise common to analog home-made circuits. Smart lighting uses the Yeelight Python library [9] in favor of plain TCP scripting, as it encapsulates the LAN protocol of the device and avoids reliance on cloud services. Natural-language processing translates phrases such as "make it warmer" or "dim to half" into Yeelight function calls regardless of pre-known keywords in the language model. For video capabilities, the ESP32-CAM performs MJPEG streaming through HTTP, discovery via inspecting the ARP table, and pan control by HTTP GET.

2.4 Comparative Analysis of AI Assistant Platforms

To establish a proper positioning of ARIA with regard to current solutions, a comparison between three major categories is provided: commercial smart speakers, social robots, and DIY educational kits. Smart speakers represent a highly engineered piece of hardware, operating under very strict intent-matching within a closed ecosystem. In contrast, DIY kits are completely open, but lack the ability of reasoning provided by current large language models. ARIA occupies a middle-ground between these two approaches by combining LLM-level intelligence with complete openness. Table 2.1 summarises the comparison along six criteria.

Table 2.1: Comparative analysis of AI assistant platforms.

Criterion	*ARIA*	*Smart speakers* (Alexa, Google Home)	*Social robots* (EMO, Vector)	*DIY kits* (Otto, Petoi)
Intelligence	LLM-based reasoning via Gemini, free-form replies	Template-based intent matching with fixed slots	Pre-scripted dialogue trees	Minimal; depends on user-written code
Physical form	Pan-capable head with active camera module	Static enclosure, no motion	Mobile chassis with expressive animations	Walking or dancing body, no sensing head
Openness	Open source: firmware, dashboard, 3D model	Closed proprietary ecosystem	Proprietary hardware and software	Fully open hardware and firmware
Audio interface	Digital I2S, 16-bit PCM at 16 kHz	High-quality proprietary audio stack	Medium-quality integrated speaker	Analog output, low fidelity
Language support	English, Russian, Kazakh (dedicated Kazakh STT)	Major languages only; no Kazakh	Single pre-configured language	Defined by the user's implementation
Cloud dependency	Optional; only LLM and external APIs use the cloud	Mandatory cloud connection for every request	Mandatory cloud connection	Fully offline

The evaluation is based on the following criteria: first, *intelligence* - ARIA is able to perform reasoning using an LLM (Gemini) and generate free-form responses; smart speakers use template-based intent matching with fixed slots; social robots follow pre-scripted dialogue trees; DIY kits provide the minimum level of intelligence written by the user. Second, *physical form* - ARIA features a pan-capable head and a camera module; smart speakers have a static enclosure; social robots have a mobile chassis with animation; DIY kits usually come with a walking or dancing body but no head with sensors. Third, *openness* - all firmware and dashboard software components are distributed as open-source projects; smart speakers run in a closed, proprietary ecosystem; social robots are completely proprietary; DIY kits are open in terms of hardware and firmware. Fourth, *audio interface* - ARIA supports high-quality audio output through the digital I2S interface (16-bit, 16 kHz PCM); smart speakers use a proprietary, high-quality

audio stack; social robots have an average audio stack; DIY kits use the lowest-quality audio output, implemented via analog interface. Fifth, *language support* - ARIA is natively equipped with English, Russian, and Kazakh speech recognition engines, including a special STT model for Kazakh; smart speakers support only major languages, excluding Kazakh; social robots support only one preconfigured language at once; DIY kits support languages according to the implementation made by the user. Sixth, *cloud requirements* - ARIA needs the cloud connection only for LLM access and API integration, being capable of fully local operation for everything else; smart speakers and social robots require a cloud connection for each request; DIY kits have no cloud dependencies.

Summing up the evaluation, smart speakers employ intent-based classification with fixed slots; however, they do not provide a possibility for free-form generation to end-users and cannot be opened to inspection due to closed ecosystems [28, 29]. Social robots such as EMO and Vector offer expressive mobile bodies but employ pre-scripted dialogue with a limited depth of reasoning [30]. DIY kits provide absolute openness but lack intelligence, leaving all responsibilities to the users. Thus, ARIA combines features from both sides and takes a middle ground: LLM-based reasoning through Gemini, full openness of the firmware and dashboard, a pan-capable head with an active camera as a substitute for a humanoid chassis, a high-quality digital audio path (I2S, 16-bit, 16 kHz PCM), native support for English, Russian, and Kazakh (including Kazakh-specific STT), and hybrid cloud/local operation - a combination that was never offered in a single product before.

Chapter 3

Design and methodology

3.1 Methodology

To guide the ARIA project, we adopted the principles of Agile development [31] by adopting iterations reminiscent of Scrum [32]. ARIA entailed three tightly coupled systems, namely embedded hardware, back-end software, and AI, and requirements kept changing throughout the project depending on the discovery of new constraints in each realm. For example, it was not until testing the camera that the decision for two ESP32 modules came into effect due to GPIO constraint [6], while using the Kazakh Speech-to-Text API via Mangisoz as a fallback option became necessary due to Whisper's failure to handle Kazakh voices [10, 11]. Thanks to short iterations, the testing of each subsystem independently, from Flask server [2] to the ESP32 audio streamer [6] to the LLM pipeline [14], was possible before bringing all components together, which made system integration easier since failure in one part of the pipeline would not affect others. A waterfall development strategy could not have worked, considering that hardware limitations, rate limitation in API [24], and real-time audio requirements had to be discovered through testing.

3.2 System overview

Figure 3.1 is the summary of the current architecture.
There is a one server(laptop) running a Flask server that processes whole logic, and optionally leverages an NVIDIA GPU via CUDA for hardware acceleration. Meanwhile browser client lets users interact with the server via HTTP and Socket.IO. ARIA devices - ESP32-WROOM and ESP32-CAM are not directly connected to the browser client. The ESP32-WROOM streams microphone PCM over UDP to the server and plays back TTS PCM received on a second UDP port. The ESP32-CAM streams MJPEG video and accepts HTTP commands to control pan(can be moved 360 degrees). Home device Yeelight bulb is controlled over the LAN connection using the manufacturer's local LAN protocol [9]

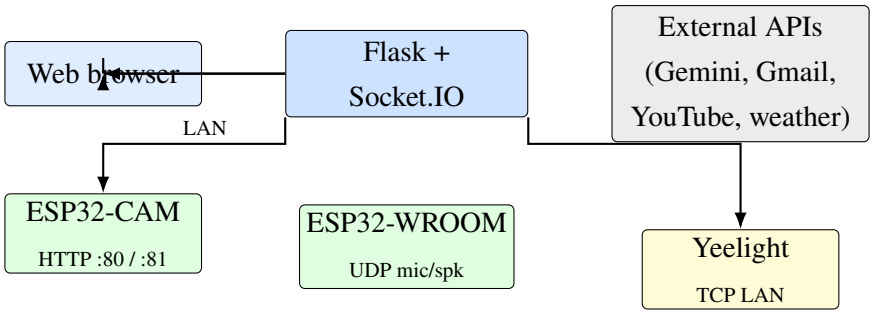

Figure 3.1: ARIA architecture (browser, gateway, peripherals, cloud APIs).

3.2.1 Software Architecture (UML Class Diagram)

To demonstrate the inner architecture of the ARIA system, the following class diagram of UML is provided (see Fig. 3.2). It should be noticed that the software consists of four logical levels as follows: (i) the controller level that involves the FlaskApplication and SocketIOHandler and provides REST and WebSocket APIs for the application; (ii) the service level that incorporates the business logic and communication with external services (GmailService, AppBulbController, ExternalAPIClient, VoicePipeline); (iii) the persistence level that represents five ORM classes (ChatMessage, User, Session, GmailAccount, EmailMessage); (iv) the hardware-bridge layer which represents two ESP32 devices as well as the Yeelight bulb using specific network protocols such as UDP, HTTP, and Yeelight LAN protocol. It should be stressed that the browser widget, YouTubePlayer, is represented by an aggregated part of WebBrowserClient, and the search functionality corresponds to the Flask route "/api/play-music" while playing the video relies on the YouTube IFrame Player API.

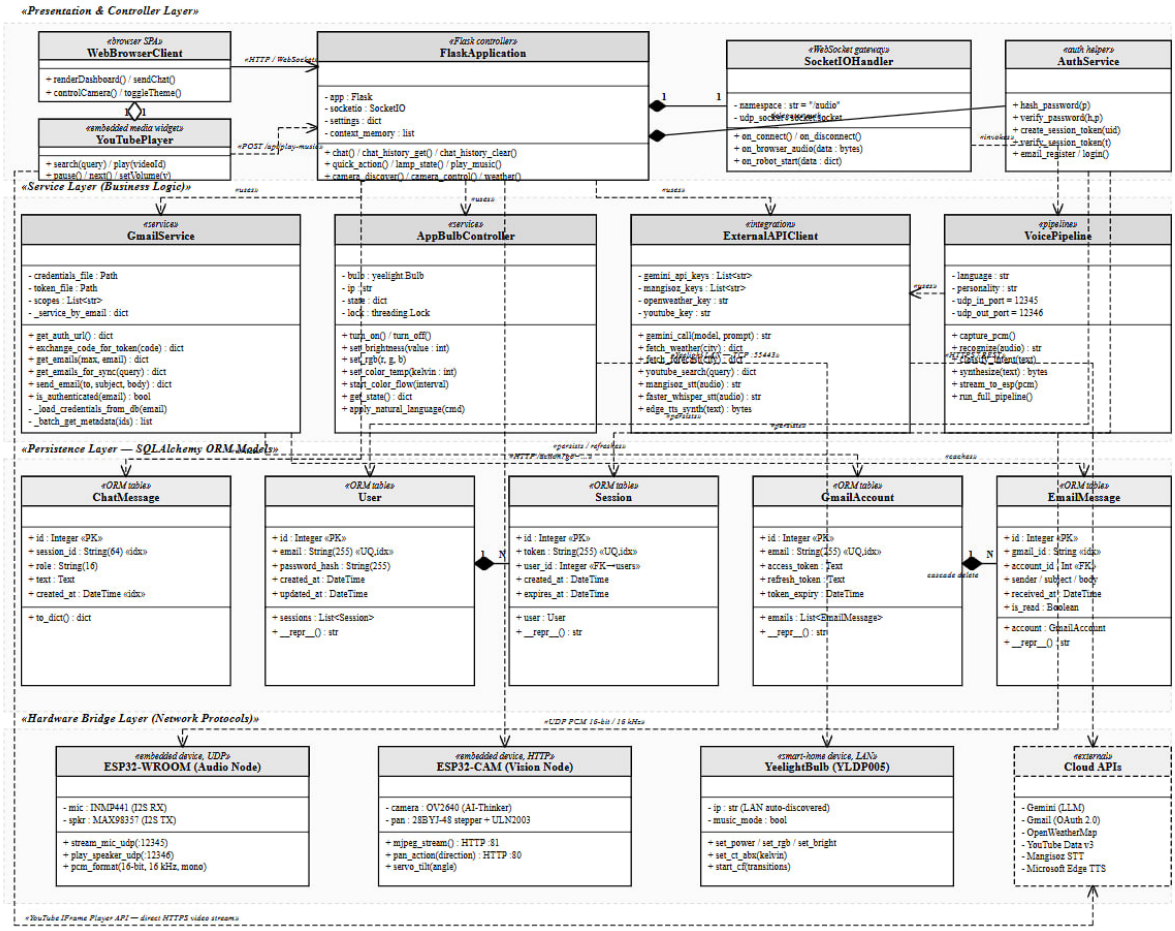

Figure 3.2: UML class diagram of the ARIA system: controller, service, persistence, and hardware-bridge layers.

It should be highlighted that in the provided diagram the dash-arrows represent the dependency between the controller and external service classes (FlaskApplication controller does not perform the tasks itself but relies on the corresponding external services). As for the diamonds, these symbols represent the composition of classes where the Flask application uses only one SocketIOHandler and every entity in User or GmailAccount models stores the corresponding Session or EmailMessage entities, respectively. Such structuring allows conducting effective unit testing because different components can be isolated and tested independently (without HTTP actions or without the running web server).

Github, and reproducibility Code of the project, including device's 3D model and ESP32 codes are maintained in a Git repository:
https://github.com/TheSuxarick/aria.

Flask Web Framework is used for initialization of the application with SQLite database set to be used locally. The implementation of the SocketIO ensures that there is real-time interaction between the client and the server. Initialization of the Gmail is also done for the email feature. (See Listing 3.1)

Listing 3.1: Flask Application Initialization and Database Configuration

```
app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///aria.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db.init_app(app)
socketio = SocketIO(app, cors_allowed_origins="*", async_mode="
    threading")
gmail_service = GmailService()
```

These database models define the structure of the persisted application data. The ChatMessage model stores every chat message associated with a specific session, while the User model stores the credentials used for logging into the system. (See Listing 3.2)

Listing 3.2: Database Models for ChatMessage and User Entities

```
class ChatMessage(db.Model):
    __tablename__ = 'chat_messages'

    id = db.Column(db.Integer, primary_key=True)
    session_id = db.Column(db.String(64), nullable=False, index=
        True)
    role = db.Column(db.String(16), nullable=False)  # "user" | "
        assistant"
    text = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)

class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(255), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
```

3.2.2 Database Schema (DBMS Design)

For the sake of explaining the persistent data layer of the ARIA architecture, the entity–relationship diagram below is provided (see Fig. 3.3). All of the application state information is stored inside the SQLite database file aria_email.db and managed by the SQLAlchemy ORM framework.

The database schema includes tables that belong to three different functional domains: (i) the Authentication & Conversation Domain, which includes users, sessions, and chat_messages tables; (ii) the Gmail Integration Domain, which involves the gmail_accounts and email_messages tables; and (iii) the Application Services Domain, represented by the music_tracks, context_memory, and app_settings tables. One can observe that the users table acts as the centerpiece of the whole schema, with all other seven tables referencing it directly or indirectly (via Gmail accounts) via foreign-key relationships.

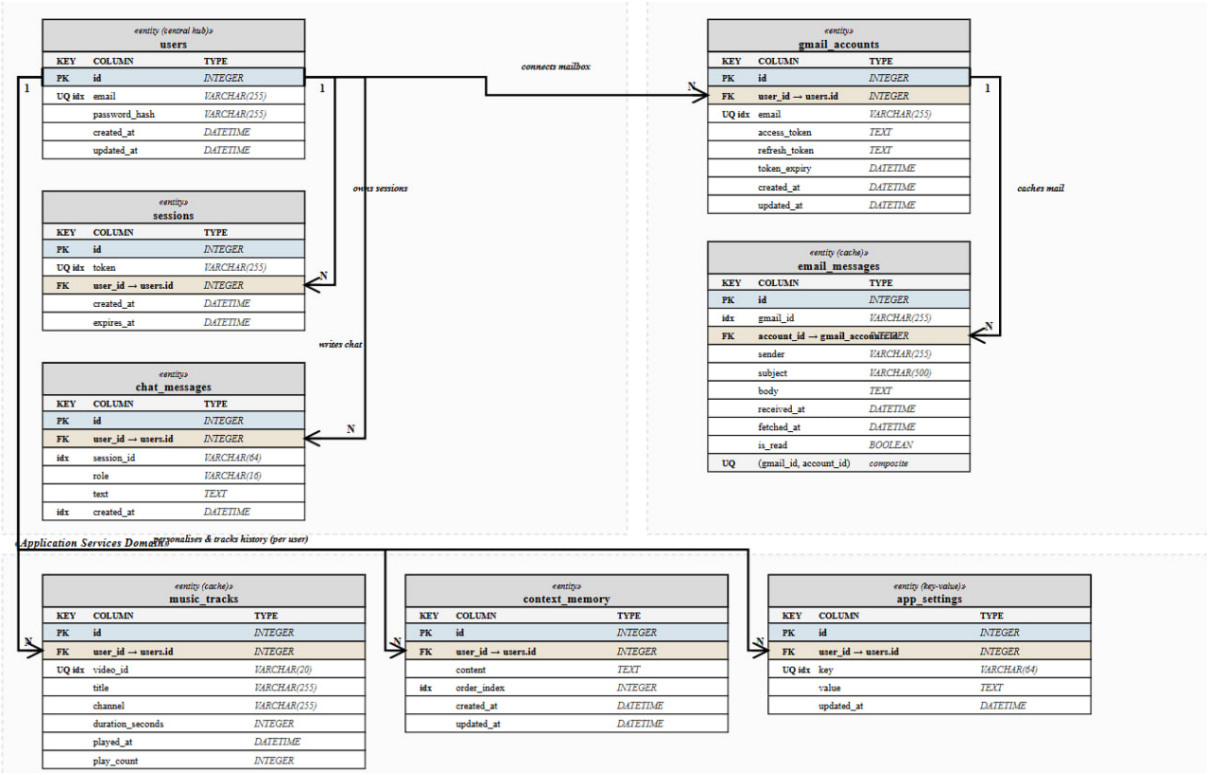

Figure 3.3: Entity–relationship diagram of the ARIA database (aria_email.db): eight entities, seven foreign-key relationships, Crow's-foot notation.

The "one" part of the relationship is denoted in the diagram by a vertical line, while the "many" part is shown by a three-legged crow's foot icon, following the IE/Barker notation. Light-blue rows show primary keys (PKs), while light-tan rows are foreign keys (FKs). Therefore, the starting and ending points of each relationship can be easily traced by examining columns visually. As for cardinalities, all relationships in the schema have a one-to-many cardinality type: from one record in users there are multiple records in sessions, chat_messages, gmail_accounts, music_tracks, context_memory, and app_settings tables, respectively; similarly, from one record in gmail_accounts, there are multiple email_messages records. Additionally, the relationship between the tables gmail_accounts and email_messages features cascade-delete behaviour, meaning that the deletion of a mailbox deletes all of its messages from cache automatically.

3.3 Hardware and physical design

3.3.1 Physical appearance

The design is inspired by popular culture robot aesthetics such as bb8, but implemented and created as an original Maya model. Printed in matte black filament, with layer lines and visible slight faceting.
Visually it consists of two segments, one larger "body" and second smaller upper "head". The head segment has the camera aperture: a circular front housing with the ESP32-CAM lens that slightly glow red, which doubles "device live" cue for demos and photography.

3.3.2 CAD geometry and internal layout

Before printing, the shell was modeled in Maya software.
Both figures below 3.4 and 3.5 show orthographic-style views of the outer surfaces.
Figure 3.6 presents a see-through (wireframe) illustrating internal architecture of model more clearly: a vertical neck region reserved for pan mechanics, body of device for the ESP32-WROOM stack, INMP441, MAX98357, and power distribution, and the rectangular cutout in the head aligned with the camera module. Body part actually consists of bottom and upper segments, for easy access to the inner electronics part of device.

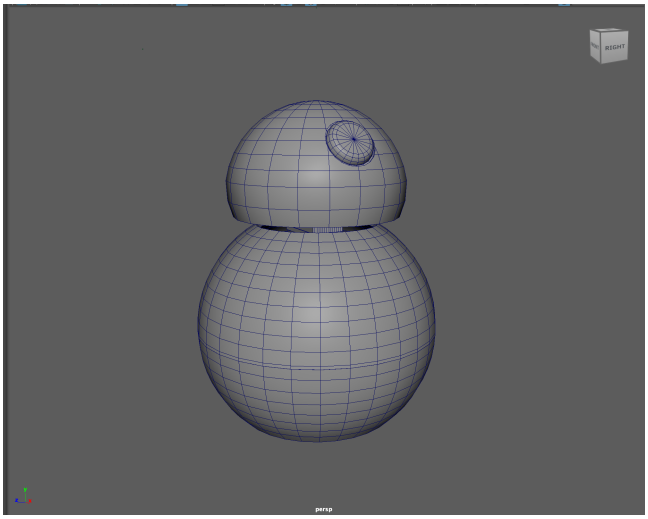

Figure 3.4: 3D model: front aspect showing head dome and camera recess.

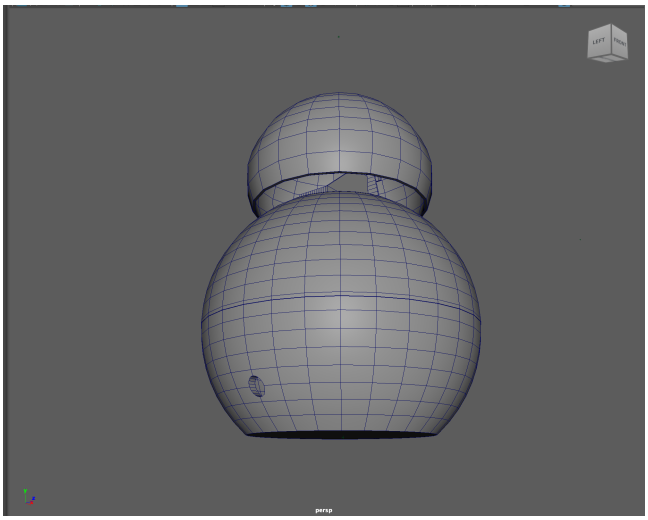

Figure 3.5: 3D model: bottom and rear regions, cable exit and inside of head dome could be observed.

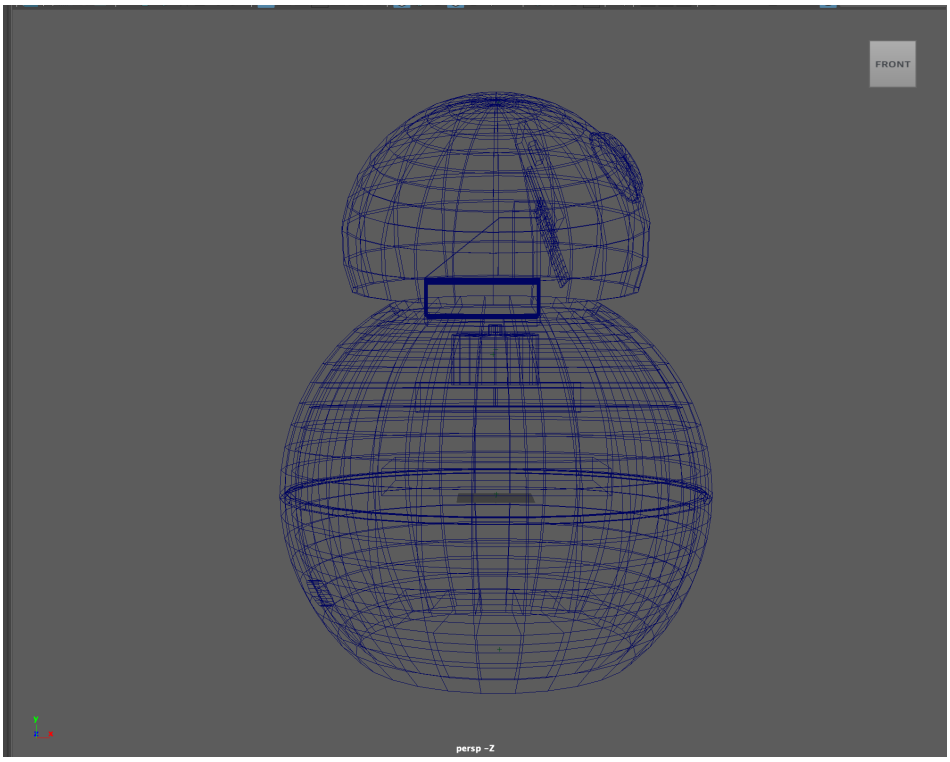

Figure 3.6: Transparent/wireframe 3D depicting internal volumes: camera pocket in the head, neck/pan region, and body shelves for audio MCU and power.

3.3.3 Rationale for two ESP32 modules

Current architecture uses one ESP32-CAM and one ESP32-WROOM dev board instead of a single MCU. One of the motivations behind it was GPIO exhaustion: the AI-Thinker camera module dedicates the most of useful pins to the parallel camera data bus, I^2C SCCB, synchronization lines, and PSRAM-friendly routing. Adding additional two independent I2S peripherals (microphone

RX and speaker TX) each requires BCLK, LRCLK/WSEL, and data lines, plus stable DMA service - and overlapping those resources with continuous camera DMA and dual HTTP servers increases interrupt latency and debugging difficulty.

More formally, this split follows practice of:

- *Pin budget and contention.* Tables 3.1-3.2 show that the OV2640 parallel interface alone occupies many GPIOs on the CAM board; remaining free pins are not enough for a clean dual-I2S audio alongside stepper and servo outputs without resorting to risky pin sharing or stripping camera features.
- *Real-time isolation.* MJPEG encoding and Wi-Fi TX on the camera MCU run continuously during streaming. Full-duplex audio on the same chip would share the RF stack and cache behavior. Separating audio to a second ESP32 isolates dropouts, a camera watchdog reboot does not necessarily mute the microphone bridge.
- *Firmware simplicity.* Each sketch stays small and testable: `esp-32-cam.ino` handles vision and motion; `esp-32-wroom.ino` handles only I2S and UDP. Easier debugging and testing and flashing code.
- *Power and mechanical packaging.* Physically, the head hosts the camera board while the body hosts audio matching the 3D intent in Figure 3.6. The prototype is powered from a 5 V, 10 A DC supply, covering inrush and simultaneous load from two ESP32 modules, the class-D amplifier peaks, stepper coils, and conversion losses.

3.3.4 ESP32-CAM module, electrical interconnect

The firmware used is `CAMERA_MODEL_AI_THINKER`. Tables 3.1 and 3.2 list GPIO assignments as defined in `esp-32-cam.ino` code.

Table 3.1: ESP32-CAM (AI-Thinker): parallel camera interface and control signals (from firmware).

Function	*Signal / role*	*GPIO*
Camera power down	PWDN	32
Camera reset	RESET	not used (-1)
External clock	XCLK	0
SCCB data / clock	SIOD / SIOC	26 / 27
Pixel data	Y2 … Y9	5, 18, 19, 21, 36, 39, 34, 35
Frame sync	VSYNC	25
Line valid	HREF	23
Pixel clock	PCLK	22

Table 3.2: ESP32-CAM (AI-Thinker): actuators and motion (project wiring in firmware).

Function	*Description*	*GPIO*
Stepper winding A1	`STEPPER_PIN1`	13
Stepper winding A2	`STEPPER_PIN2`	15
Stepper winding B1	`STEPPER_PIN3`	14
Stepper winding B2	`STEPPER_PIN4`	2
Tilt servo PWM	`SERVO_1`	12

Figure 3.7 provides the visual project wiring drawing in Miro, for easier camera assembly.

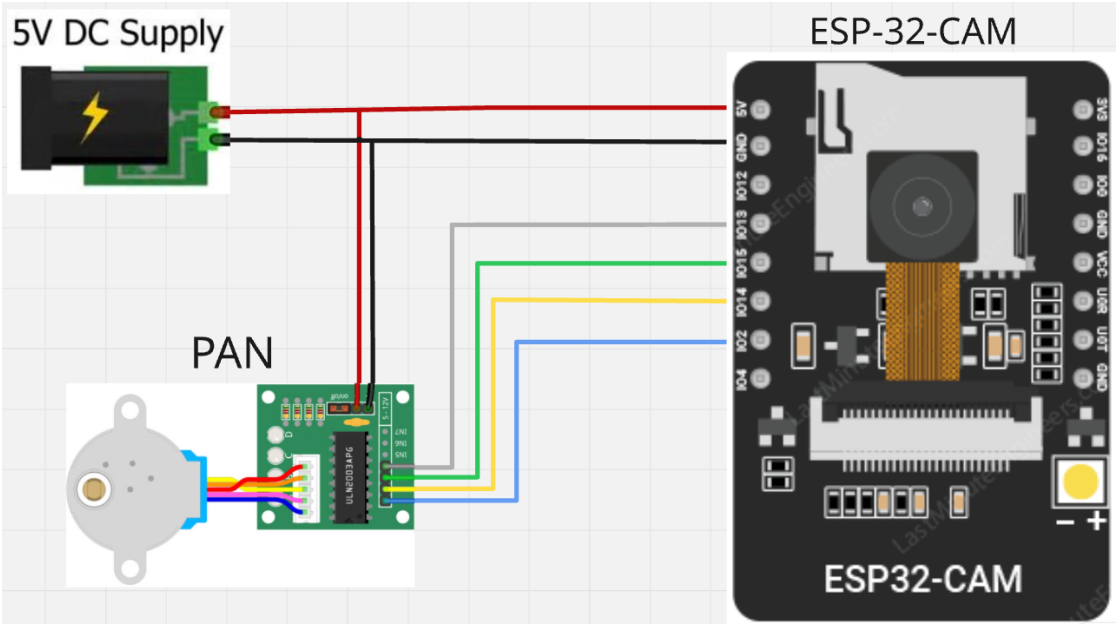

Figure 3.7: Wiring diagram for the ESP32-CAM assembly (camera module, pan stepper).

The function of this component is receiving commands from the Frontend and passing them through an HTTP request to the ESP32 controller in the local area network. It allows for remote control of camera motion in the ARIA system. (See Listing 3.3)

Listing 3.3: Camera Control Endpoint for ESP32 Communication

```
@app.route("/api/camera/control", methods=["POST"])
def camera_control():
    data = request.get_json()
    direction = data.get("direction", "")
    if direction not in ("left", "right"):
        return jsonify({"error": "Invalid direction"}), 400

    requests.get(f"http://{_camera_ip}/action?go={direction}",
       timeout=3)
```

```
9     return jsonify({"status": "ok", "direction": direction})
```

3.3.5 ESP32-WROOM audio module, electrical interconnect

The audio bridge uses two I2S peripherals: RX for the INMP441 and TX for the MAX98357. Table 3.3 lists GPIOs as implemented in repository firmware (`esp-32-wroom.ino`). GPIO 34 is input-only and is used for the microphone data line per the prototype wiring.

Table 3.3: ESP32 (WROOM) I2S pin assignment in repository firmware.

Function	*Signal*	*GPIO*
INMP441	Bit clock (SCK / BCLK)	26
INMP441	Word select (WS / LRCLK)	25
INMP441	Serial data (SD)	34
MAX98357	Bit clock (BCLK)	12
MAX98357	Left-right clock (LRC)	14
MAX98357	Data in (DIN)	27

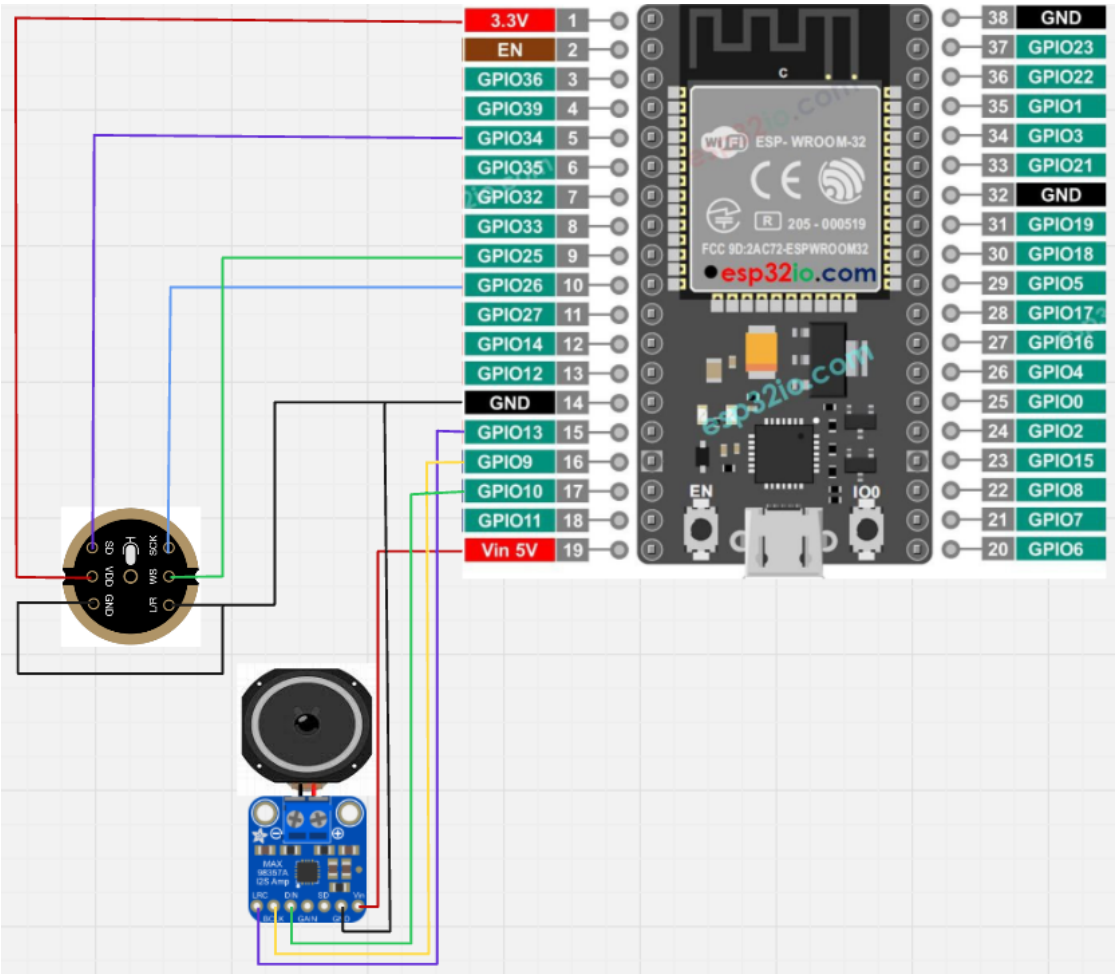

Figure 3.8: Wiring diagram for the ESP32-WROOM audio node (INMP441 + MAX98357).

Audio protocol. Both directions use 16-bit signed PCM, mono, 16 kHz, little-endian, over UDP. The microphone path sends datagrams to `PC_IP:12345`; the PC sends speaker frames to the ESP on port `12346`.

This function looks into the local network ARP table to determine the IP address of the ESP32 unit based on the MAC address. This method ensures that devices can reconnect automatically even when the IP address changes due to dynamic allocation by the router. (See Listing 3.4)

Listing 3.4: Network Device Discovery by MAC Address

```
def _find_ip_by_mac(target_mac):
    output = subprocess.check_output(["arp", "-a"]).decode("utf-8")
    target_mac_clean = target_mac.lower().replace("-", ":")

    for line in output.splitlines():
        if target_mac_clean in line.lower():
            match = re.search(r"(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d
                {1,3})", line)
            if match:
```

```
                return match.group(1)
    return None
```

3.4 Web Dashboard: User Interface Design

Once all the hardware components were put together, the process of designing the web control panel began. The objective was to ensure that the user interface for the ARIA dashboard appeared more professional, like an actual commercial product instead of a rudimentary prototype created by engineers. All design decisions regarding color scheme, fonts, space between elements, and form factor were made to improve usability in the context of a smart home environment.

3.4.1 Design Philosophy

The interface uses a SPA architecture, meaning there's a Flask backend server [2] rendering just one HTML page, and JavaScript takes care of switching between Dashboard, Camera, Functions, and Settings views seamlessly, without reloading an entire webpage each time. This solution was selected due to the necessity of immediate visual feedback in smart home controllers, where performing full HTTP requests to render a page each time after pressing a button would be perceived as slow, especially when compared to native applications.

The main interface style follows glassmorphism and neo-dark trends popular among dashboards right now. Cards appear against an ever-so-slightly textured background, and there are emerald radial glow effects as well as borders emitting colored light. At the same time, there's a light theme variant of the interface available which can be toggled with one click of a button and stored in localStorage.

3.4.2 Colour Analysis and Palette

Color is the first thing that grabs your attention when interacting with an interface; for ARIA, color performs three functions: creating visual hierarchy, indicating system state, and minimizing eye strain.

The dark design palette consists of a slate blue-grey background color (0f172a). This color makes the design more comfortable compared to pure black because it reduces the harshness. It is perfect for designing controllers used in dark environments since that is where smart home devices are typically installed. The card designs and sidebars use a slightly lighter slate grey color (1e293b). Accent color, on the other hand, uses a brighter green color (10b981) that contrasts well with dark backgrounds. Its selection is based on the association between active states and bright green colors.

Table 3.4 lists the complete dark theme palette as defined in the CSS.

Table 3.4: ARIA dark-theme colour palette (CSS custom properties).

Token	*Value*	*Role*
`-bg-primary`	`#0f172a`	Page background (deep slate)
`-bg-secondary`	`#1e293b`	Sidebar, cards, secondary surfaces
`-bg-input`	`#0f172a`	Input fields (matches page)
`-accent`	`#10b981`	Primary accent: buttons, links, active indicators
`-accent-light`	`#34d399`	Hover states, logo gradient
`-accent-glow`	`rgba(16,185,129,0.35)`	Box-shadow glow on focus/hover
`-accent-bg`	`rgba(16,185,129,0.12)`	Tinted backgrounds for badges
`-text-primary`	`#f1f5f9`	Headings and body copy
`-text-secondary`	`#94a3b8`	Labels and descriptions
`-text-muted`	`#64748b`	Placeholders and disabled text
`-success`	`#22c55e`	Success states (online, connected)
`-warning`	`#f59e0b`	Warnings (low signal, quota near limit)
`-danger`	`#ef4444`	Errors (offline, failed requests)
`-info`	`#38bdf8`	Informational badges

Light theme uses a soft, pale colors:

- Background: near-white (f8fafc)
- Cards: pure white (ffffff)
- Text: dark slate (0f172a) for strong contrast
- Accent: deeper green (059669) so it stands out on light backgrounds
- Shadows: subtle slate (rgba(15,23,42,0.07))

Table 3.5 shows the overridden tokens for the light theme.

Table 3.5: ARIA light-theme colour overrides.

Token	*Value*	*Change vs. dark theme*
`-bg-primary`	`#f8fafc`	Near-white page background
`-bg-secondary`	`#ffffff`	Pure white cards and sidebar
`-bg-input`	`#f1f5f9`	Soft gray input fields
`-accent`	`#059669`	Deeper emerald for light-bg legibility
`-accent-light`	`#10b981`	Swapped with dark-theme primary accent
`-text-primary`	`#0f172a`	Dark slate for high readability
`-text-secondary`	`#475569`	Medium slate for labels
`-text-muted`	`#94a3b8`	Lighter muted tone
`-success`	`#16a34a`	Slightly deeper green
`-warning`	`#d97706`	Deeper amber
`-danger`	`#dc2626`	Deeper red
`-info`	`#0284c7`	Deeper sky blue

3.4.3 Layout and Component System

Design includes the presence of a fixed left bar which is 240 px wide, containing the logo, navigation elements, and server status. The content area occupies the remaining part of the screen space and begins with a sticky header measuring 64 px, which consists of the current page name, theme toggler, and server status. Under the header, each page will have an uninterrupted scroll experience.

Cards are used as the key elements for design. They include a gradient semi-transparent background color, a 1 px emerald-colored border, rounded corners measuring 16 px, and lift-on-hover behavior.

Component shapes follow a *border-radius scale*:

- 8 px buttons, input field, badge;
- 12 px modals, dropdown menu;
- 16 px cards, the main content container;
- 50% avatar circles, status.

3.5 Product User Flow

The central communication concept behind ARIA involves the speech pipe line; the user communicates verbally with the physical machine and receives a coordinated reply using synthetic speech and movement from the head and body. Refer to Figure 3.7.

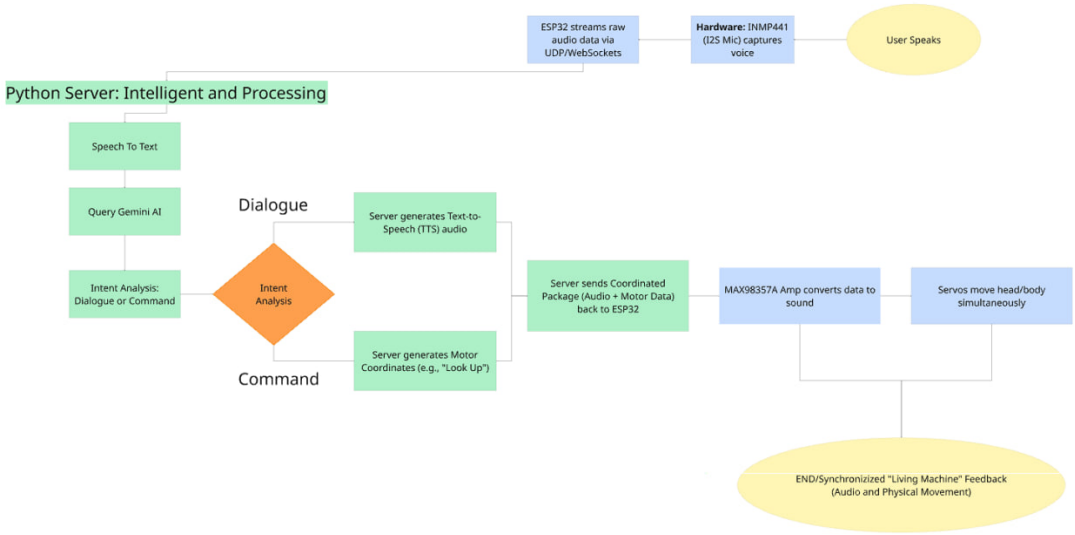

Figure 3.9: Product user flow: voice interaction pipeline from user speech to synchronised audio and physical feedback.

The flow proceeds through the following stages.

Stage 1: Voice Capture

The process begins with the user communicating with the ARIA device. The INMP441 MEMS microphone, which uses I2S as its interface, samples the audio data at 16 kHz with 16-bit precision. The ESP32-WROOM module reads the content of the I2S RX buffer through DMA and immediately converts the sampled PCM data into UDP packets for transmission to the Python server on PC IP:12345.

Stage 2: Speech-to-Text

At the Python server-represented by the green area “Intelligent and Processing” in Figure 3.7—input audio data is analyzed using the Speech-to-Text engine. Depending on the language identified or selected, one of the following engines will be used:

- *Mangisoz STT* - engine for Kazakh speech;
- *Faster-Whisper* [11] - engine for Russian and English, using CTranslate2 INT8 quantisation for low-latency CPU inference;
- *Gemini-based recognition* [14] - secondary fallback when Mangisoz is unavailable.

Stage 3: LLM Query

The transcript is uploaded to the Gemini AI engine. The prompt will be made up of the transcription, the current personality selection, context memory entries, and if applicable, emails from cached Gmail entries.

The chat endpoint processes all the incoming messages that come through from the frontend side. They get validated and saved into the database before being forwarded to a suitable service handler, such as controlling devices, getting information or running the Gemini AI model. The reply generated will be saved again and returned to the frontend. (See Listing 3.5)

Listing 3.5: Chat API Endpoint with Message Processing and Routing

```
@app.route("/api/chat", methods=["POST"])
def chat():
    data = request.get_json()
    user_message = data.get("message", "")
    sid = _get_chat_session_id()

    if not user_message:
        return jsonify({"error": "Empty message"}), 400

    _save_chat_msg(sid, "user", user_message)

    ai_text, err = _gemini_call(model, system_text, contents)
    _save_chat_msg(sid, "assistant", ai_text)
    return jsonify({"reply": ai_text})
```

Stage 4: Intent Analysis

Once the LLM responds to the query, intent analysis occurs on the server to determine whether the intent was either a Dialogue (reply to the query) or a Command (action on the device). Intent classification is illustrated by the orange diamond node in Fig 3.9. Both branches do not exclude each other since an utterance could activate both branches simultaneously.

- *Dialogue branch.* The server uses Microsoft Edge TTS [12] to render the textual reply provided by the LLM into speech. The output PCM stream serves as the audio segment of the coordinated reply.
- *Command branch* The server determines the appropriate motor coordinates for the action specified in the command. If the command asks for some movement of the servos or steppers, such as “Look up,” the server calculates the corresponding angle or offset of the

stepper. In case the command pertains to the remote device (e.g., “Turn the lamp blue”), it is translated to the appropriate Yeelight API call [9].

Stage 5: Coordinated Package Delivery

Both outputs from the dialogue and command are packaged as one single unit, which is a combination of audio and motor commands. The packaged outputs are then sent to the ESP32 through UDP packets (audio on port 12346). Packaging of both the data ensures that they arrive at the device simultaneously.

Stage 6: Synchronised Output

On the ESP32 part, two events take place in parallel:

1. The *MAX98357A* Class D Audio Amplifier takes in PCM audio input from the I2S TX, transforms it into analog signals and makes them reach the speaker for the user to hear the response from the assistant.
2. The *stepper motor and tilt servo*synchronize movement of the head and body with the audio.

3.5.1 Web Dashboard Navigation

In addition to the voice pipeline described above, the user interfaces with ARIA through a web dashboard. The single-page app provides four views that can be reached from the sidebar menu.

1. *Dashboard.* The main menu provide three cards:
    - *Smart Lamp* - power swithc, brightness slider, temperature presets, eight colour dot presets, and an RGB button. Every interaction sends a `POST /api/quick-action` request; the lamp state badge updates in real time via a `lamp_update` Socket event [7].
    - *Voice Robot* - presents an option of pressing just one button to launch the voice pipeline mentioned earlier. The UI shows status updates ("Listening…", "Thinking…", "Speaking…") throughout the stages.
    - *AI Chat* - text-based conversation with AI chat. Chat history is erasable and can be cleared. Transcriptions from the voice robot also appear.
2. *Camera.* Navigating here triggers automatic ESP32-CAM discovery via ARP-table inspection (`GET /api/camera/discover`). If found, the MJPEG stream loads into an `<img>` tag. A directional D-pad sends pan/tilt commands (`POST /api/camera/control`). A microphone button enables two-way audio: the browser streams to the ESP32 speaker via Socket.IO [8], and a "Listen" button streams device audio back.
3. *Functions.* Three utility services in a grid:

- *Music Player* - searches YouTube via the Data API [5] and plays music.
- *Weather* - fetches current conditions and a five-day forecast from OpenWeatherMap [3], refreshing every 180 seconds.
- *Email* - displays Gmail inbox after OAuth 2.0 authentication [13, 25]. Users connect accounts via a modal, synchronise with a "Sync" button, and read individual messages in a detail overlay.

4. *Settings.* Personalisation and system setting in a two-column grid:
   - *Context Memory* - stored entries injected into LLM prompts for personalised responses.
   - *Model & API Status* - active Gemini model and service health.
   - *Personality* - persona cards ("Friendly", "Professional").
   - *Language* - English / Russian / Kazakh
   - *Wake Word* - toggle activation.
   - *Audio Source* - selector between ESP32 and PC audio devices.

The Gmail application fetches the OAuth credentials from the database corresponding to the specific user account. If the access token expires, it will automatically be renewed by means of the refresh token stored on the database. (See Listing 3.6)

Listing 3.6: Gmail OAuth Credential Loading and Token Refresh

```
def _load_credentials_from_db(self, email: str):
    acct = GmailAccount.query.filter_by(email=email).first()
    if not acct or not acct.access_token:
        return None

    creds = Credentials(
        token=acct.access_token,
        refresh_token=acct.refresh_token,
        token_uri='https://oauth2.googleapis.com/token',
    )
    if creds.expired and creds.refresh_token:
        creds.refresh(Request())
        acct.access_token = creds.token
        db.session.commit()
    return creds
```

The sendMessage function handles the user interaction through the chat interface, displaying the message in the chat window and sending it to the API backend service. The message returned by the server is then displayed by the function on the user interface. (See Listing 3.7)

Listing 3.7: Frontend Send Message Function

```
async function sendMessage() {
    const text = chatInput.value.trim();
    if (!text) return;

    addChatBubble("user", text);
    chatInput.value = "";
    showTyping();

    const resp = await fetch("/api/chat", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ message: text })
    });
    const data = await resp.json();
    removeTyping();
    addChatBubble("assistant", data.reply);
```

Chapter 4

Results and Discussion

4.1 Functions

Currently our system supports:
1) Both browser-based and voice chat with Gemini LLM of Google in ENG/RU/KZ languages
2) Reading and summarizing cached Gmail messages when OAuth is configured
3) Weather forecast queries
4) Playing YouTube videos/music by natural language
5) Toggling and recoloring a Yeelight bulb using keyword rules in multiple languages
6) Remote control of the camera, including the ability to speak and hear through the speaker and microphone.
7) AI personality behavior change.

4.2 Demonstration scenarios

To validate the ARIA system works as intended end-to-end, the team did several *scenario scripts* (informal acceptance tests):

1. *Morning briefing.* User says "Computer" and asks for the weather forecast for today in the Russian language. ARIA replies back with information taken from API. The same process works when asked from chat.

2. *Lighting demo.* In the dashboard of website user sets warm white color of lamp at 80%, after that asks in chat to switch to blue light and start to dim. Lamp state badge updates consistently.

3. *Inbox triage.* User asks for a summary of the latest newsletter from his gmail, the model uses cached `EmailMessage` rows without requesting the full thread from Google for every token.

4. *Remote control and presense*User opens the camera page to interact with the person that is near camera, sees MJPEG video on page driven by LAN, pans left/right to track a subject, and successfully uses microphone and speaker of esp-32-wroom for communication.

5. *Fallback execution* Try to disconnect ESP32-CAM power. UI of website shows empty state and toast (Figure 4.4). Disconnect from the Internet. Chat degrades with explicit error rather than silently failing, those helping user to understand error is in the internet connection.

Scenarios above are not a formal user study with N participants. They are engineering walk-throughs of system's capabilities that has been thoroughly tested

4.3 Final prototype appearance

Figure 4.1 shows the assembled final ARIA device with the completed physical enclosure, camera placement, and overall product form achieved at the end of implementation.

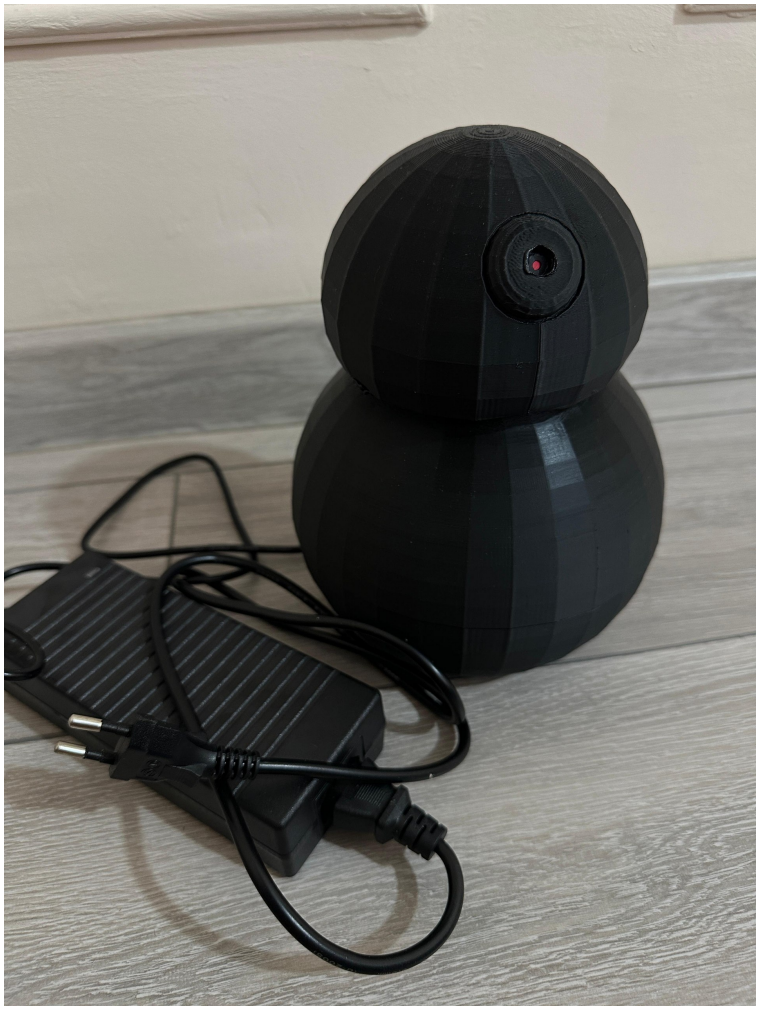

Figure 4.1: Final assembled ARIA device prototype used in demonstration scenarios.

The following figures present screenshots captured from the ARIA dashboard. Both dark and light themes are shown to demonstrate the dual-theme implementation.

4.3.1 Dashboard Page

The Dashboard page forms the primary landing interface of the ARIA control panel by combining the three most used interactions on one screen - left-aligned smart lamp control, one-tap voice robot launcher on the upper right side, and live AI chat panel on the lower right corner. Figures 4.2 and 4.3 show the same page design implemented using two different themes as specified above.

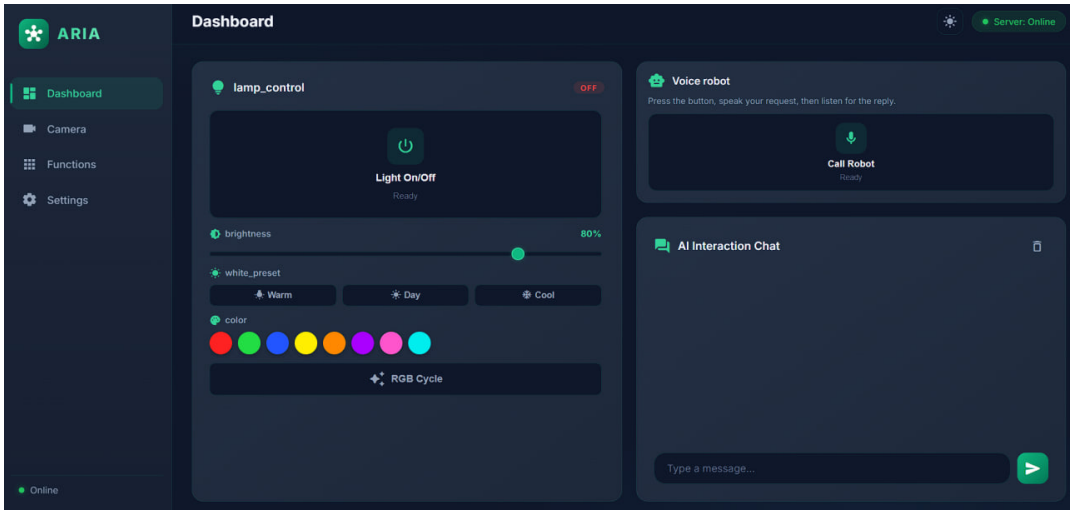

Figure 4.2: Dashboard page in dark theme: lamp controls (left), voice robot activation (top right), and AI chat (bottom right).

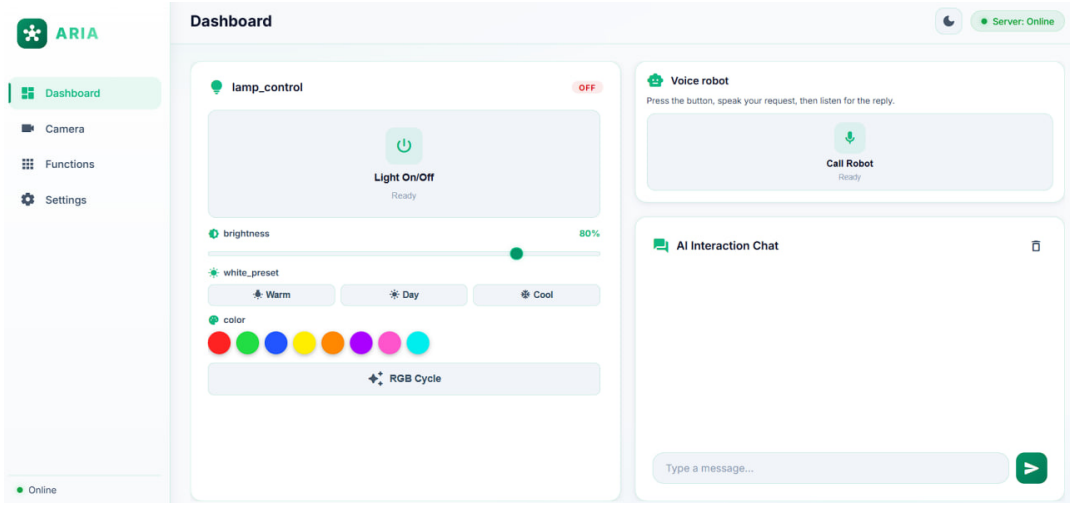

Figure 4.3: Dashboard page in light theme, demonstrating the alternate colour scheme.

4.3.2 Camera Page

The Camera page opens up whenever the user seeks to observe and interact with the region ahead of the ARIA system in real-time. After ESP32-CAM detection and initialization, the MJPEG live feed from the device is displayed in the center of the page, while the directional D-pad below the feed allows the user to rotate the camera either to the left or right direction using the stepper motor. Figure 4.4 shows how the page looks like when live video streaming takes place.

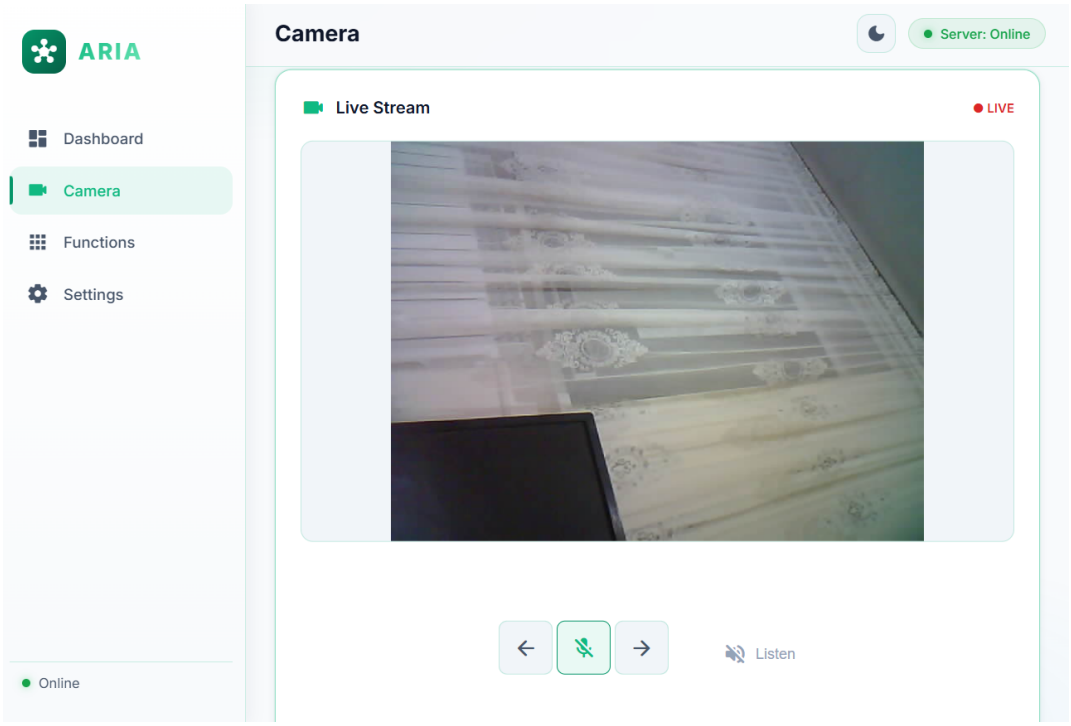

Figure 4.4: Camera page showing the live MJPEG video feed from the ESP32-CAM and the directional control pad.

4.3.3 Functions Page

The Functions page groups all the utility functions that come standard with ARIA together, including the YouTube music player, OpenWeatherMap forecast widget, and Gmail mailbox. These utilities are placed together for the convenience of the user to monitor the weather forecast, play music, and check for the latest emails without navigating away from the dashboard. Figures 4.5 and 4.6 demonstrate how the page looks when the music player, weather widget, and email reader are filled with current data.

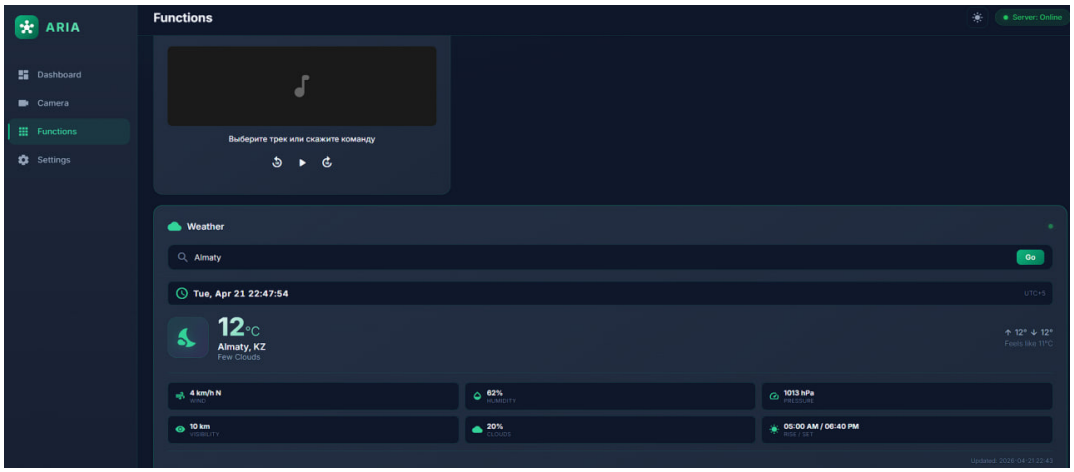

Figure 4.5: Main Functions page: Music and Weather interface.

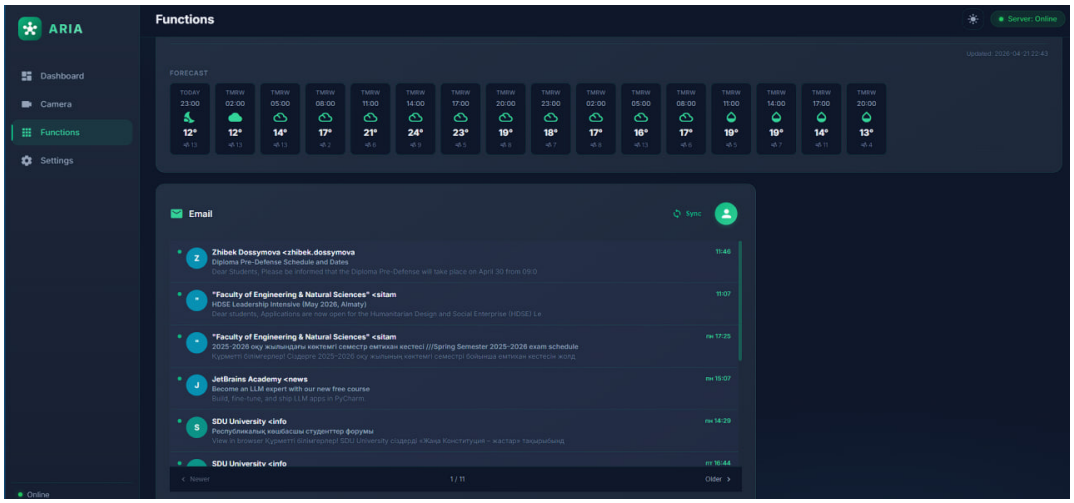

Figure 4.6: Main Functions page: Weather and email interface.

4.3.4 Settings Page

The Settings page acts as the point where the user can modify the behavior of ARIA and configure the system settings. Here, the user can view and edit the context memory used to supplement prompts to the large language model, the personality selection menu that influences the assistant's tone, change the interface language between English, Russian, and Kazakh, and pick the audio source responsible for transferring voice commands and responses between the ESP32 device and PC. Figure 4.7 demonstrates the whole structure of this page.

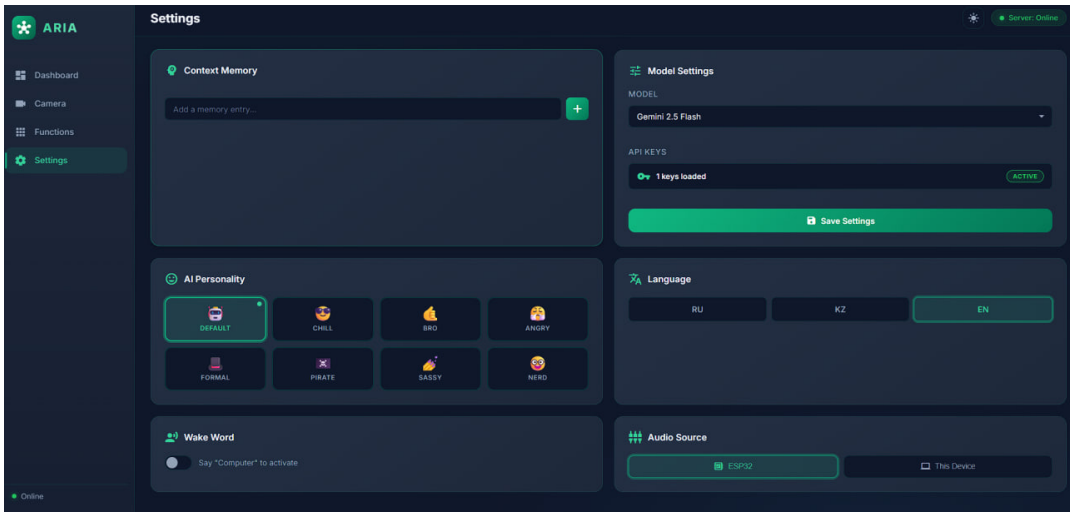

Figure 4.7: Settings page: context memory (top left), personality selector (top right), language and audio source options (bottom).

4.4 Performance across multiple languages

To provide quantitative evidence of the voice pipeline speed, we measured the time it took to process each of the three language scenarios (English, Kazakh, Russian). Table 4.1 reports the recorded values from live robot runs.

Table 4.1: Voice pipeline timing by language (measured on prototype).

Language	*STT (s)*	*LLM (s)*	*TTS gen (s)*	*Processing (s)*	*Total wall (s)*
English	0.66	1.37	1.45	3.47	12.05
Kazakh	1.43	3.90	2.67	8.00	23.03
Russian	0.72	1.51	1.65	3.87	12.30

The measurement above show stable operation of the full multimodal chain (STT $\rightarrow$ LLM $\rightarrow$ TTS) in all three supported languages. English and Russian responses are generated with near-identical interaction speed, while Kazakh language was harder to process for the system. In all cases, the pipeline consistently completes and returns synchronized voice output to the ESP32 device.

4.5 BOM and cost-oriented summary

Table 4.2 is the summary of the functional role of each hardware component. It depicts the engineering contribution of each part and complements the detailed cost comparison in the next subsection.

Table 4.2: Hardware component roles in the ARIA prototype.

Component	*Functional role*	*Interface*
ESP32-CAM (AI-Thinker)	Camera capture, MJPEG streaming, pan command handling	HTTP, Wi-Fi
ESP32-WROOM Dev Board	Audio bridge between robot and server (voice input/output)	UDP, Wi-Fi, I2S
INMP441 microphone	Speech capture from user to processing pipeline	I2S RX
MAX98357 + speaker	Voice playback from ARIA responses	I2S TX
Stepper motor + driver	Horizontal head/camera movement (pan axis)	GPIO digital control
3D-printed enclosure	Physical integration of modules into final device body	Mechanical assembly
Yeelight YLDP005 bulb	Smart-home lighting endpoint for control scenarios	LAN protocol (TCP)
Flask host PC/laptop	Central server: STT, LLM, TTS, APIs, dashboard backend	HTTP/WS, local runtime

4.5.1 Prototype cost comparison (Kazakhstan vs. China sourcing)

Table 4.3 compares the local and Chinese market prices for prototype hardware components to evaluate the economic feasibility of producing the ARIA device. Local pricing data was sourced from Kaspi.kz, while Chinese market prices were obtained from Pinduoduo. Foreign currency conversions are based on the National Bank of Kazakhstan's official exchange rate of 68.82 KZT/CNY as of April 20, 2026. [33]

Table 4.3: Cost comparison of components for prototype assembly.

Component	*Kazakhstan (KZT)*	*China (CNY)*
INMP441	1650	6.2
MAX98357 (I2S)	1500	12.44
Jumper wires	756	5.0
ESP32-CAM	4715	33.0
ESP32-WROOM	3500	26.8
Stepper motor	1230	7.7
Subtotal (electronics only)	*13351*	*91.14(CNY)≈6272(KZT)*
3D-printing body/material (approx.)	2000(KZT)	2000(KZT)
Total without delivery	*15351*	*≈8272 KZT*
Estimated total with delivery	-	*≈10000 KZT*

4.5.2 Yeelight lamp used in tests

Control of light was validated on a consumer Yeelight bulb with local LAN API support. Table 4.4 is characteristics of model that has been used during testing. Other Yeelight models of bulbs that have the same LAN protocol should be compatible too, but only the YLDP005 model was confirmed in the lab.

Table 4.4: Yeelight smart bulb used for ARIA lighting tests.

Attribute	*Value*
Product name	Smart LED Bulb W3 (Multicolor)
Manufacturer model	YLDP005
Mains supply	100-240 V, 50/60 Hz; rated current 0.07 A
Optical output	900 lm nominal
Color temperature	1700-6500 K (white range); multicolor (RGB) modes
Rated lamp power	8 W

Chapter 5

Conclusion

The main contributions of this thesis are the introduction of ARIA, an intelligent multimodal smart home assistant based on a Flask web service, cloud AI capabilities, and an ESP32-based physical device. ARIA features voice control, messaging, live video stream and control, automated smart lights, weather forecast, music player, email utility, and multi-language support for English, Kazakh, and Russian.

The goals of the research were achieved using a modular approach to building the system with two ESP32 boards (for cameras and audio) and real-time communication through Socket.IO and UDP connections, alongside a user dashboard. The system works flawlessly end-to-end, with multiple tests of the whole voice pipeline with three languages, and a successful demo showcased.

Firstly, the purpose of the current research was to resolve fragmentation problems associated with closed ecosystems of contemporary smart homes. To do this, researchers had decided to develop a single open-source system that could be used as an intelligent personal AI assistant called ARIA. The objective was met with the help of efficient integration of latest achievements in artificial intelligence, which included a usage of the new large language model, Gemini, and physical hardware elements.

As mentioned above, the proposed design was made using an Agile approach, which allowed researchers to build a modular architecture. The use of intelligent distributing of computing load, when vision and motion were processed with the help of the ESP32-CAM module while the audio signal was sent to the ESP32-WROOM module in the format of I2S, proved that microcontrollers were capable of handling complex tasks efficiently. In terms of the system architecture, the central element of ARIA was a Flask web server, which facilitated interaction with the help of Socket.IO and UDP protocols among the user's web dashboard, the robotic itself, and different cloud APIs.

Another evidence of successful implementation of ARIA's architecture is its capability to operate in several modes as well as in different languages. Namely, the use of special modules to translate speech into text, including Mangisoz for Kazakh and faster-Whisper engines for Russian and English, allowed to implement the functionality without problems. Indeed, empirical testing proved that voice control and sound capture, as well as recognition of intents and corresponding motor feedback, worked successfully in all three languages. Moreover, it was possible to test

common functionalities of the system, like turning a Yeelight smart bulb on/off or adjusting its brightness, playing an audio track from YouTube, or logging in to an email account through OAuth authentication.

Economically, the proposed robotics could be considered as an alternative way to build a quality conversational assistant at a reasonable price, amounting to $20. The availability of completely transparent software code and high-quality audio allows developing a good alternative to commercial solutions.

Though the current development cannot be regarded as a complete product, there are several ideas for further improvements, namely, adding more IoT protocols, improving the print quality of the device, and applying edge computing technology. Despite this, ARIA proves to be a good example of convergence between language models and IoT.

In conclusion, ARIA is a viable example of an implementable prototype that demonstrates the integration artificial intelligence and Internet of Things technologies into a working home assistant system.

Bibliography

[1] *Global Smart Home Market 2025-2029 (press summary).* Technavio. https://newsroom.technavio.org/smart-homemarket-v2

[2] *Flask Documentation.* Pallets Projects. https://flask.palletsprojects.com/en/stable/

[3] *Weather APIs.* OpenWeather. https://openweathermap.org/api

[4] *Gmail API Overview.* Google. https://developers.google.com/gmail/api/guides

[5] *YouTube Data API v3.* Google. https://developers.google.com/youtube/v3

[6] *Get Started - ESP32 - ESP-IDF Programming Guide.* Espressif Systems. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/index.html

[7] *Flask-SocketIO Documentation.* Miguel Grinberg. https://flask-socketio.readthedocs.io/en/latest/

[8] *Socket.IO Documentation (v4).* https://socket.io/docs/v4/

[9] Korokithakis, S. *YeeLight library (Python).* https://yeelight.readthedocs.io/

[10] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I. *Robust Speech Recognition via Large-Scale Weak Supervision.* arXiv:2212.04356, 2022. https://arxiv.org/abs/2212.04356

[11] *faster-whisper.* SYSTRAN. GitHub repository. https://github.com/SYSTRAN/faster-whisper

[12] *edge-tts.* PyPI package. https://pypi.org/project/edge-tts/

[13] *Using OAuth 2.0 to Access Google APIs.* Google. https://developers.google.com/identity/protocols/oauth2

[14] *Gemini API Documentation.* Google. https://ai.google.dev/gemini-api/docs

[15] OpenAI. *GPT-4 Technical Report.* arXiv:2303.08774, 2023. https://arxiv.org/abs/2303.08774

[16] Touvron, H., Lavril, T., Izacard, G., et al. *LLaMA: Open and Efficient Foundation Language Models.* arXiv:2302.13971, 2023. https://arxiv.org/abs/2302.13971

[17] Lewis, P., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* arXiv:2005.11401, 2020. https://arxiv.org/abs/2005.11401

[18] *pydub.* James Robertson. GitHub repository. https://github.com/jiaaro/pydub

[19] *Django Documentation.* Django Software Foundation. https://docs.djangoproject.com/

[20] *FastAPI Documentation.* Sebastián Ramírez. https://fastapi.tiangolo.com/

[21] *SQLite Documentation.* https://www.sqlite.org/docs.html

[22] *SQLAlchemy 2.0 Documentation.* https://docs.sqlalchemy.org/en/20/

[23] *Flask-SQLAlchemy Documentation.* Pallets Projects. https://flask-sqlalchemy.palletsprojects.com/en/stable/

[24] *Flask-Limiter Documentation.* Ali-Akber Saifee. https://flask-limiter.readthedocs.io/en/stable/

[25] Hardt, D. *The OAuth 2.0 Authorization Framework.* RFC 6749, IETF, 2012. https://datatracker.ietf.org/doc/html/rfc6749

[26] *Requests: HTTP for Humans.* https://requests.readthedocs.io/en/latest/

[27] *python-dotenv.* PyPI package. https://pypi.org/project/python-dotenv/

[28] Hoy, M. B. *Alexa, Siri, Cortana, and More: An Introduction to Voice Assistants. Medical Reference Services Quarterly*, 37(1):81-88, 2018. https://doi.org/10.1080/02763869.2018.1404391

[29] Purington, A., Taft, J. G., Sannon, S., Bazarova, N. N., Taylor, S. H. *"Alexa is my new BFF": Social Roles, User Satisfaction, and Personification of the Amazon Echo. Proceedings of the 2017 CHI Conference Extended Abstracts*, pp. 2853-2859, ACM, 2017. https://doi.org/10.1145/3027063.3053246

[30] Breazeal, C., Dautenhahn, K., Kanda, T. *Social Robots.* In *Springer Handbook of Robotics*, pp. 1935-1972, Springer, 2016. https://doi.org/10.1007/978-3-319-32552-1_72

[31] Beck, K., Beedle, M., van Bennekum, A., et al. *Manifesto for Agile Software Development.* 2001. https://agilemanifesto.org

[32] Schwaber, K., Sutherland, J. *The Scrum Guide.* 2020. https://scrumguides.org/scrum-guide.html

[33] *Exchange Rates of National Bank of Kazakhstan.* https://nationalbank.kz/en/exchangerates/ezhednevnye-oficialnye-rynochnye-kursy-valyut/graph?beginDate=13.04.2026&endDate=20.04.2026&rates%5B0%5D=8